

Pivot Summary Performance Review

Scope and headline

I reviewed the uploaded `app.py` and `db_core.py` files. I have **not changed either file**. This is an architectural/performance review, not a code patch.

The pivot already has several worthwhile optimizations: it aggregates in PostgreSQL rather than loading raw transactions into pandas; it caches pivot source data and built outputs; it combines grouping statistics so averages are computed correctly; it has a cached grouping layer for subtotals; it uses an optimized row-tree builder; and it virtualizes the visible grid. Those are good foundations.

For a selection with about **seven Rows fields, two Columns fields, and several Values fields**—for example monthly amount columns such as March, April, and November—the most likely remaining costs are:

1. **One full aggregation query per Values field.** Each query groups by the same Rows/Columns dimensions and scans the same filtered fact table again.
2. **A large output shape.** Seven row levels create many subtotal rows; two column fields create many leaf and subtotal columns. The result can have a large number of cells even when the aggregated source table is much smaller than the raw table.
3. **Multiple products built from that data.** The app builds the flat pivot, the styled Summary Matrix, the Field Efficiency table, and formatted Excel download bytes; some of these happen automatically on the pivot page.
4. **Rendering/export work is still full-size.** AgGrid virtualizes browser DOM rows, but the app first creates records for the complete DataFrame. Excel serialization also loops over the table's cells.

So the likely explanation for “slow but correct” is cumulative work across repeated scans, repeated per-value pivots, full matrix construction, rendering, and exports—not necessarily an incorrect or wholly unoptimized aggregation algorithm.

What the current implementation does well

- **Pushes grouped aggregation to PostgreSQL.** `fetch_pivot_source_data` in `db_core.py` groups by the selected dimensions and computes sum/count/min/max in SQL, rather than transferring every raw row to Python (approximately lines 435–518).
- **Keeps enough statistics to support multiple aggregation choices.** The sum/count/min/max statistics allow correct subtotal and grand-total calculations,

including mean as sum divided by count (approximately lines 418–430 and 521–554).

- **Avoids rescanning the aggregate table for every cell.** `_make_dims_cache` groups once per distinct dimension combination and reuses those grouped results (approximately lines 597–645).
- **Avoids an older expensive recursive row-tree approach.** `_build_row_entries` ranks unique values and walks sorted combinations (approximately lines 735–816).
- **Uses caching.** The Streamlit app caches pivot source data for 10 minutes and pivot-building functions and formatted Excel bytes for 5 minutes (`app.py` lines 177–191).
- **Uses a virtualized grid for on-screen display.** This reduces browser DOM work for rows not currently in view (`app.py` lines 572–618), though it does not eliminate all full-result processing.
- **Has a row-render guard.** It estimates subtotal rows and warns above 3,000 rows (`app.py` lines 2453–2471). This protects against one known failure mode, but it does not limit the total number of cells or columns.

These are good choices. I would preserve their correctness and build on them rather than replacing the pivot engine wholesale.

Likely bottlenecks, in priority order

1. Separate database scan for every Values field — highest-priority opportunity

In `app.py` (around lines 2415–2427), the app calls `fetch_pivot_source_data` once for each selected Values column. Each call builds a query over the same table with the same filters and grouping fields; only the numeric value column changes. The calls are made sequentially.

The cache is keyed separately for each value column, so it can help on a later rerun for an identical selection, but it does not combine the first-run work. With several monthly measure columns, PostgreSQL may perform several nearly identical scans and group-bys.

Improvement idea: have the database aggregation produce statistics for all selected Values fields in one grouped query. Keep one row per selected Rows/Columns combination and include sum/count/min/max for each measure. This should reduce repeated scans and round trips, especially when the fact table is large. It also avoids rebuilding the same grouping keys separately in Python for each value field.

Correctness detail: maintain a distinct count for each Values field, since the fields may have different null patterns. Mean must remain sum divided by that field's non-null count.

Grand totals and subtotals must be recombined from the statistics, not from a mean of means.

2. Grouping/output cardinality — often the next limiting factor

The source query groups on all selected Rows and Columns fields. As the number of dimensions and their distinct values increase, the number of distinct combinations can grow rapidly. The summary then adds a subtotal row at each Rows depth and subtotal columns for the Columns hierarchy.

Seven selected Rows fields do not simply mean seven columns in the output. The number of displayed rows is approximately the sum of distinct prefixes at every row depth, plus a grand-total row. Two Columns fields can also create many leaf columns and subtotal columns. With multiple Values fields, the displayed width is multiplied by the number of measures. Date fields in the summary expand into Month and Date levels, which adds hierarchy/header work even when the underlying unique dates are the same.

Improvement idea: make the output shape a first-class limit. Estimate both row count and column count, then estimate total cells before building the full matrix. Warn or ask the user to filter/reduce dimensions when the estimated output is too large. The existing 3,000-row test should be supplemented with a cell/column limit; a short but extremely wide pivot can still be expensive.

For users, the most effective immediate practices are:

- Apply selective Filters before using many hierarchy fields.
- Use only the Rows levels needed for the question; move detail fields out of Rows if they are not required.
- Avoid a high-cardinality field (such as a transaction ID or customer ID) in a summary intended to show totals.
- Limit the number of Values and Columns fields when a broad, exploratory result is acceptable.

3. The app builds more than the main Summary Matrix

Once the pivot has been built, the app constructs a flat pivot (`build_pivot_table` , `app.py` around lines 2473–2476) and later serializes it to Excel (around lines 2810–2821). It then builds the styled Summary Matrix (around lines 2490–2514), serializes a formatted workbook for it (around lines 2516–2527), and also builds a Field Efficiency table by default (around lines 2649–2689), followed by another Excel export (around lines 2795–2805). Optional Flow Retention and DPD outputs add more work.

The flat view uses the first Values field, while the Summary Matrix and Field Efficiency table can process every selected Values field. The output is correct, but a user who only wants

the Summary Matrix pays for additional views and download artifacts too.

Improvement idea: make the essential Summary Matrix the default work. Make Field Efficiency, Flow Retention, and alternate flat exports explicitly opt-in or generate them only when requested. Generate Excel bytes on demand rather than during every page evaluation, if the UI framework allows that interaction. The app should not build an expensive secondary table simply because the user is looking at the primary summary.

4. Full matrix computation and formatting remain proportional to output size

`build_excel_style_pivot` uses cached groupings, which is good, but it still iterates over every display row and every display column to fill the matrix (`db_core.py` around lines 959–996). That means work and memory still scale with the number of output cells, and the work repeats for each Values field.

The AgGrid path creates metadata and a records representation from the complete DataFrame before displaying it (`app.py` around lines 594–618). Virtualization avoids mounting every visible row in the browser at once; it does **not** avoid constructing/serializing the complete result. The Excel writer also loops through each data cell and formats/writes it (`db_core.py` around lines 1114–1263). A large workbook can therefore be slow even when the database query is fast.

Improvement idea: for larger pivots, render a manageable subset or an expandable hierarchy rather than transmitting every subtotal/detail row immediately. Keep the full export as an explicit user-requested operation. A total-cell budget is more meaningful than row count alone. If complete export is needed, consider a separate, clearly indicated export step so users understand that it is a separate large operation.

5. Streamlit’s “Build Summary” button does not permanently freeze the selection

The button sets a session flag indicating a pivot was built (`app.py` around lines 1920–1932). After that, the code says the pivot stays live and reflects current selections. Streamlit reruns the script when widgets change. Existing caches should help for unchanged selections, but changing Rows, Columns, Values, or Filters changes cache keys and can trigger the full workload again. The flat `build_pivot_table` call is also not wrapped in the app’s cache decorators, so it is repeated on reruns while the pivot branch executes.

Improvement idea: decide whether the intended product behavior is “live update after first build” or “only recompute after the user clicks Apply/Build.” For large pivots, an explicit Apply/Build workflow is usually more predictable. Store the last-applied selection and only run the expensive pipeline when it changes through an explicit action. Also cache

the complete computed result, including the flat summary if retained, using a stable selection key plus a data freshness/version strategy.

Cache caution: the current cached database source has a 10-minute TTL. That improves repeat interactions but can show data that is up to that old unless refreshed. A production design should make freshness visible and offer a deliberate refresh/invalidation mechanism rather than relying on an undocumented cache expiry.

6. PostgreSQL query plan and indexing

The grouped query must read enough source rows to calculate correct totals. Indexes cannot make an unfiltered full-table aggregation free. They are most useful when users apply selective filters, or when a date/time range meaningfully limits the scan.

Improvement idea: inspect the real generated query with PostgreSQL's execution plan before adding indexes. Use `EXPLAIN (ANALYZE, BUFFERS)` on a representative slow selection and check:

- how many source rows are read and how many groups are returned;
- whether the plan uses a sequential scan, index scan, or bitmap scan;
- whether hash aggregation or sorting spills to disk;
- time spent reading buffers versus aggregating;
- whether filters are selective and applied early.

Then add indexes only where the plan and common workload justify them. For selective categorical filters, indexes on frequently filtered columns—and sometimes composite indexes aligned with common filter combinations—may help. For date-range filtering, use a query form that allows an ordinary date index to be used. Avoid creating a very wide index over every pivot dimension by default: it can increase storage and slow inserts/updates, and it may not help a full scan/group-by.

For repeated reports over a large, mostly stable fact table, consider a maintained summary/materialized view for common dimension/measure combinations. That can turn repeated full aggregation into reading a compact pre-aggregated relation. It requires a clear refresh/freshness policy and is less suitable for arbitrary ad hoc dimensions.

Recommended implementation sequence

Phase 1 — Measure one representative slow case

Record the exact table, filters, selected Rows, Columns, Values, and aggregation function. Capture these counts and durations separately:

1. Number of database queries and elapsed time per query.
2. Number of rows in each aggregated source DataFrame.
3. Estimated/displayed subtotal row count and number of output columns.
4. Total output cells for the Summary Matrix and Field Efficiency table.
5. Time to build the flat pivot, Summary Matrix, and Field Efficiency result.
6. Time to create each Excel workbook and to convert/send the grid records.
7. PostgreSQL execution-plan row counts, buffer reads, and spill information.

This separates a **database scan bottleneck** from a **wide/high-cardinality pivot bottleneck** and an **export/render bottleneck**. Without this split, it is easy to optimize the database and still see little improvement because the full output is the dominant cost.

Phase 2 — Reduce duplicated work with minimal behavior change

1. Combine the per-Values SQL aggregations into one grouped query.
2. Avoid building Field Efficiency and flat/export variants until they are requested.
3. Add a total-cells/column-size guard alongside the existing subtotal-row guard.
4. Make the build/apply behavior explicit so a widget interaction does not unexpectedly launch a large recalculation.
5. Reuse the computed result for the UI and exports wherever possible.

This is likely the best first implementation phase because it targets repeated work without changing the definition of the pivot.

Phase 3 — Optimize the database for measured workloads

1. Use actual execution plans to identify whether filters can benefit from indexes.
2. Refresh table statistics if they are stale.
3. Address aggregation spills only when the plan shows them; memory settings should be tuned cautiously and preferably per workload/session rather than globally.
4. For recurring reports, evaluate a pre-aggregated/materialized summary with a defined refresh schedule.

Phase 4 — Scale the user experience for genuinely large pivots

1. Use a cell-based output threshold, not only a row-based threshold.
2. Show compact summaries first; reveal detail levels on demand where possible.
3. Generate full Excel downloads only when explicitly requested.

4. For very large datasets, consider server-side pagination/group expansion or a dedicated asynchronous export process rather than sending/rendering the entire table in one Streamlit run.
-

Validation checklist before accepting an optimization

Performance changes must preserve existing semantics. Compare old and new results on a fixed sample for:

- Sum, count, min, max, and mean.
- Null values in different measure columns (counts must remain measure-specific).
- Grand totals and every row/column subtotal.
- Two or more Values columns with different missing-value patterns.
- Date fields at month and day levels.
- Master hierarchy rows with no matching data, if that feature is enabled.
- Filtered and unfiltered cases.

Measure cold-cache and warm-cache runs separately. A warm cache can make a slow query appear fixed while the first real user run remains slow.

Bottom line

For the described case, I would investigate in this order: **(1) repeated database scans per Values field, (2) the total number of output cells created by seven Rows plus two Columns and multiple Values, (3) automatic construction/export of the extra Field Efficiency and flat views, and (4) rerun/cache behavior.** The database-side aggregation is already a strong design choice; the highest-value next step is to eliminate repeated scans and avoid generating secondary outputs unless they are needed, then optimize indexes or pre-aggregation based on real query plans.

These are informed hypotheses from static code review. Actual timing breakdowns and PostgreSQL plans are needed to establish which component dominates for your data and hardware.